

Guia de Developer do ATC

■# Guia de Developer do Advanced Trigonometry Calculator

Este guia da uma visao tecnica de alto nivel do codigo do ATC. O codigo-fonte continua a ser a referencia final para comportamento exato.

Para receitas de contribuicao, checklists e notas praticas de "como adicionar", ver `docs/pt-PT/Developer_Reference.md`.

Objetivo do projeto

O ATC e uma aplicacao matematica C++ de linha de comandos para Windows. Avalia expressoes textuais e comandos documentados em ambiente de consola.

O projeto suporta fluxos numericos com aritmetica, trigonometria, numeros complexos, matrizes, estatistica, ferramentas polinomiais, resolucao de equacoes, variaveis e precisao configuravel.

O ATC nao deve ser descrito como um CAS geral completo.

Estrutura de codigo

Diretorio principal:

`Advanced Trigonometry Calculator/`

Ficheiros importantes:

- `main.cpp`: arranque, settings e escolha entre `double` e `Boost mp_float`;
- `auto_complete.cpp`: edicao interativa, autocomplete e historico;
- `main_aux_processor.cpp` e `main_processor.cpp`: routing de comandos e expressoes;
- `processing_core.cpp`: processamento de expressoes;
- `commands.cpp`: comandos visiveis ao utilizador;
- `equation_solver.cpp`, `equations_system_solver.cpp` e `solver.cpp`: caminhos de solver;
- `polynomial_arithmetic.cpp`: simplificacao polinomial;
- `dynamic_allocations.cpp`: helpers de memoria dinamica.

Fluxo de processamento

Para uma visao visual, consultar `docs/Architecture.md`.

1. `main.cpp` inicializa paths e settings. 2. O runtime tipado e escolhido para `double` ou `Boost mp_float`. 3. O comando entra no fluxo principal. 4. `main_core<T>()` e `main_sub_core<T>()` coordenam comandos, variaveis, output e expressoes. 5. As expressoes passam por `initialProcessor<T>()`, `arithSolver<T>()` e `functionProcessor<T>()` quando aplicavel.

Precisao

O modo persistente e guardado em:

`%USERPROFILE%\Pictures\Advanced Trigonometry Calculator\higherPrecision.txt`

Valores:

- 0: `double`
- 1: `Boost mp_float`

Memoria

Ao alterar código sensível a memória:

- usar helpers existentes quando apropriado;
- garantir terminação `\0` em arrays de char;
- libertar arrays temporários em todos os caminhos;
- correr testes de regressão e stress quando praticável.

Testes

Scripts principais:

- `tests/run-atc-regression.ps1`
- `tests/run-atc-memory-stress.ps1`
- `tests/run-atc-isolated-coverage.ps1`

Resultado validado atual:

Summary: 359 passed, 0 failed

Referencia pratica de developer

A Referencia de Developer cobre:

- como adicionar um novo comando;
- como adicionar uma função matemática;
- como adicionar um módulo guiado;
- como adicionar testes de regressão;
- expectativas de documentação;
- riscos de regressão no parser e solver;
- cuidados de compatibilidade Windows;
- regras de estabilidade de output;
- checklist antes de commit/release.

Usar em conjunto com este guia e com a visão de arquitetura antes de alterar parser, solver, consola ou persistência.